

EDDMail — Fase 2

Manual de Integración por Grupos

Integrantes: Jairo · Asher · Mario

Curso: Estructuras de Datos — USAC

Fecha: 05/10/2025

Índice

1. Contexto y objetivos
2. Panorama de módulos por integrante
3. Modelos de datos y mapeos
4. Arquitectura de integración
5. API de integración (capa “Glue”)
6. Flujo de casos de uso
7. Reportes y generación de DOT
8. Pasos de compilación y dependencias
9. Pruebas sugeridas
10. Riesgos, acuerdos y convenciones
11. Anexos (fragmentos clave de código)

1) Contexto y objetivos

Este manual describe cómo integrar el Árbol BST de comunidades y la lista de mensajes desarrollados por los tres integrantes dentro del proyecto EDDMail (Fase 2). El objetivo es unificar los aportes para: (a) permitir crear comunidades (root), (b) publicar mensajes en ellas, (c) recorrer/visualizar comunidades en orden alfabético y (d) generar reportes (DOT/Graphviz y listados).

2) Panorama de módulos por integrante

2.1 Jairo

Unidad: ArbolBST (Object Pascal). Implementa un BST que indexa comunidades por nombre. No define la estructura completa de comunidad; en su lugar almacena una referencia (Ref: PComunidadNodo) a un nodo proveniente de la unidad ListasListas. Expone operaciones: BST_Insert, BST_Find, BST_FillNamesInOrder, BST_CollectInOrderRefs, BST_ObtenerComunidadesOrdenadas.

2.2 Asher

Unidad: ArbolBST (Object Pascal). Es equivalente funcionalmente a la de Jairo (mismo API/estructura). La diferencia es mínima o nula; ambas se consideran la misma 'capa de índice BST' que apuntará a nodos de comunidad definidos externamente.

2.3 Mario

Unidades: ComunidadesBST y Comunidades. ComunidadesBST implementa un BST autocontenido con su propio tipo PComunidad, el cual incluye nombre, fecha_creacion, num_mensajes y una lista enlazada de PMensaje. Provee funciones para crear comunidades, publicar mensajes, hacer recorrido InOrder y generar DOT.

Comunidades implementa creación y gestión de comunidades y miembros acopladas a la UI (GTK), utilizando listas simples y funciones de utilidad (ShowError/ShowInfo, ListBoxAppend, etc.).

3) Modelos de datos y mapeos

Existen dos enfoques de 'modelo de comunidad':

- a) Modelo indirecto (Jairo/Asher): BST guarda claves (Nombre) y referencias a nodos de comunidad (PComunidadNodo) definidos fuera del BST (p.ej., en ListasListas). Los mensajes residirían en el nodo referenciado.
- b) Modelo autocontenido (Mario): BST define PComunidad con lista propia de PMensaje.

Decisión de integración: se adoptará el 'Modelo indirecto' como capa oficial de índice, porque se alinea con el enunciado de la fase (BST como índice de comunidades) y permite reutilizar estructuras existentes (listas simples de comunidades/mensajes). El código de Mario aporta

funcionalidades valiosas (Mensajes, DOT) que se incorporarán mediante una capa de adaptación ('Glue') para no duplicar datos.

Mapa de tipos propuesto:

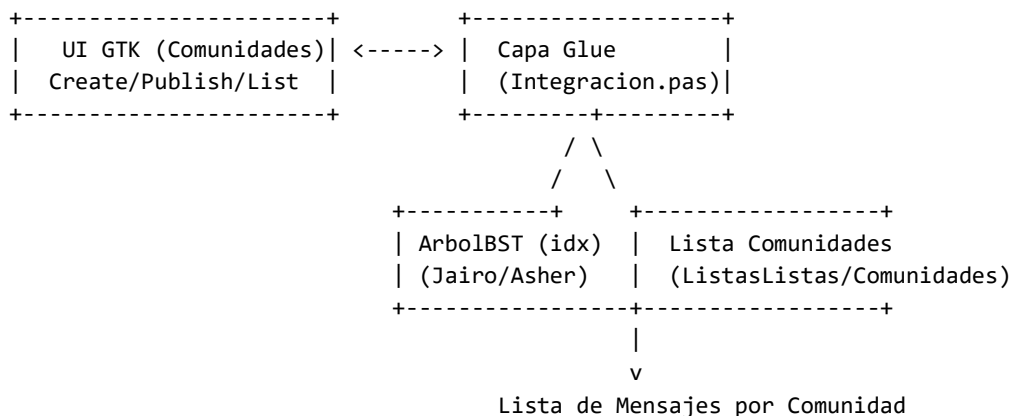
```
-- Jairo/Asher (ArbolBST) --
PBSNodo / TBSNodo
comunidades)
TBSNodo.Ref : PComunidadNodo
FechaCreacion, NumMensajes, Mensajes*)
Clave : string (Nombre comunidad)

-- Core (ListasListas / Comunidades) --
PComunidadNodo (nodo en lista de
PComunidadNodo^.Dato : (Nombre,
Nombre/Fecha/Mensajes viven en el nodo real

* Mensajes puede ser lista simple (PMensajeNodo) mantenida por Comunidades/Usuarios.
```

4) Arquitectura de integración

La arquitectura final mantiene una única fuente de verdad de comunidades (lista simple + campos Nombre, FechaCreacion, NumMensajes, Mensajes). El BST (ArbolBST) sólo indexa por nombre y almacena referencias a esos nodos. La unidad 'Comunidades' (UI/GTK) invoca funciones de alto nivel para crear comunidades, agregar miembros y refrescar vistas. La nueva capa 'Glue' coordina todas las llamadas.



5) API de integración (capa "Glue")

Se propone una unidad IntegracionComunidades.pas que centraliza las operaciones usadas por la UI y encapsula el índice BST. Esta capa asegura: creación única de comunidades, actualización consistente de contadores y publicación de mensajes.

```
// IntegracionComunidades.pas
unit IntegracionComunidades;
```

```

{$mode ObjFPC}{$H+}

interface

uses
    SysUtils, Classes,
    // Índice
    ArbolBST,          // de Jairo/Asher
    // Datos base
    ListasListas,      // define PComunidadNodo y su Dato (Nombre, FechaCreacion,
    NumMensajes, Mensajes)
    // Opcional UI/servicios
    Comunidades;       // si se requiere integración con UI y validaciones

// Inicializa estructuras (llamar al iniciar sesión root/usuario)
procedure IC_Init;

// Crea comunidad (root). Inserta en lista base y luego indexa en BST.
function IC_CrearComunidad(const Nombre, FechaISO: string): Boolean;

// Publica mensaje en una comunidad; aumenta contador y mantiene lista de mensajes.
function IC_PublicarMensaje(const NombreCom, Correo, Contenido, FechaISO: string):
Boolean;

// Llena lista de nombres ordenados (para combos/menús)
procedure IC_ListarComunidadesOrdenadas(Dst: TStrings);

// Reporte estilo líneas (Nombre | Fecha | #Mensajes)
function IC_ReporteComunidades: TStringList;

implementation

procedure IC_Init;
begin
    // Nada especial si las listas ya están inicializadas.
    // Asegurar que el BST esté vacío al reconstruir índice si recargan datos:
    // (no se expone reset en ArbolBST, pero puede hacerse reasignando raíz si se ajusta
    la unidad)
end;

function _FindComunidadEnLista(const Nombre: string): PComunidadNodo;
begin
    // Recorrer la lista global de comunidades (definida en ListasListas/Comunidades) y
    devolver puntero.
    // Esta función depende del módulo de listas base.
end;

function IC_CrearComunidad(const Nombre, FechaISO: string): Boolean;
var node: PComunidadNodo;
begin
    Result := False;
    // 1) Validar duplicado en lista base

```

```

    if _FindComunidadEnLista(Nombre) <> nil then Exit;
    // 2) Crear nodo en lista base (asignar FechaCreacion y NumMensajes:=0)
    // 3) Insertar referencia en BST
    //    BST_Insert(node);
    Result := True;
end;

function IC_PublicarMensaje(const NombreCom, Correo, Contenido, FechaISO: string):
Boolean;
var ref: PComunidadNodo;
begin
    Result := False;
    // 1) Buscar referencia en BST (O(log n))
    ref := BST_Find(NombreCom);
    if ref = nil then Exit;
    // 2) Insertar al final de la lista de mensajes del nodo ref^.Dato
    // 3) Inc(ref^.Dato.NumMensajes);
    Result := True;
end;

procedure IC_ListarComunidadesOrdenadas(Dst: TStrings);
begin
    BST_FillNamesInOrder(Dst);
end;

function IC_ReporteComunidades: TStringList;
begin
    Result := BST_ObtenerComunidadesOrdenadas;
end;

end.

```

Notas: Si se decide aprovechar la generación de DOT de Mario, se puede implementar un adaptador que recorra BST_CollectInOrderRefs y emita nodos/aristas, sin duplicar el árbol. Ver sección 7.

6) Flujo de casos de uso

6.1 Crear comunidad (root)

```

UI -> IC_CrearComunidad(Nombre, FechaISO)
1) Validación duplicados en lista base
2) Insertar nodo en lista base con (Nombre, Fecha, NumMensajes=0, Mensajes=nil)
3) BST_Insert( puntero_al_nuevo_nodo )
4) UI refresca listas: IC_ListarComunidadesOrdenadas(...)

```

6.2 Publicar mensaje en comunidad

```

UI -> IC_PublicarMensaje(NombreCom, Correo, Contenido, FechaISO)
1) ref := BST_Find(NombreCom)
2) if ref=nil => ShowError('No existe la comunidad')
3) Insertar mensaje al final de la lista ref^.Dato.Mensajes
4) Inc(ref^.Dato.NumMensajes)
5) UI: si corresponde, refrescar panel de mensajes

```

6.3 Listar comunidades ordenadas (In-Order)

```

UI -> IC_ListarComunidadesOrdenadas(Dst: TStrings)
Internamente: BST_FillNamesInOrder(Dst)

```

7) Reportes y generación de DOT

Para un reporte textual rápido, usar IC_ReporteComunidades que ya devuelve una TStringList con líneas formateadas desde el BST. Para gráficos, se sugiere implementar un emisor DOT compatible con Graphviz sin duplicar estructuras. Ejemplo de adaptador DOT sobre el índice (pseudo):

```

// Adaptador DOT usando referencias del BST
procedure IC_BST_ToDOT(const pathDot: String);
var refs: TList; i: Integer; c: PComunidadNodo; s: TStringList;
begin
  refs := TList.Create; s := TStringList.Create;
  try
    s.Add('digraph ComunidadesBST {');
    BST_CollectInOrderRefs(refs);
    for i:=0 to refs.Count-1 do begin
      c := PComunidadNodo(refs[i]);
      s.Add(Format('  n%d[label="%s (%d msgs)"]; ',
        [PtrInt(c), c^.Dato.Nombre, c^.Dato.NumMensajes]));
      // Como el índice es BST, opcionalmente añadir relaciones si se almacenan
      punteros Izq/Der en el índice
      // o bien sólo listar los nodos.
    end;
    s.Add('}');
    s.SaveToFile(pathDot);
  finally
    s.Free; refs.Free;
  end;
end;

```

Alternativamente, si se prefiere el emisor DOT ya escrito por Mario, se puede reutilizar su lógica, pero alimentándola desde el índice actual, asegurando que no existan dos fuentes de verdad.

8) Pasos de compilación y dependencias

1) Asegurar unidades en el proyecto y orden lógico:

- ListasListas.pas (o Comunidades.pas que define PComunidadNodo y lista base)
- IntegracionComunidades.pas (nueva)
- ArbolBST.pas (Jairo/Asher) – usar una sola versión para evitar conflictos
- (Opcional) módulo de Reportes DOT (si se separa)
- UI (GTK) y main

2) Incluir en uses según corresponda (SysUtils, Classes, GTK2/glib2 si hay UI).

3) Al iniciar sesión/root: llamar IC_Init; al cargar datos masivos (JSON), reconstruir índice con BST_Insert por cada comunidad creada.

4) Ubicar Graphviz en el PATH si generarán imágenes desde DOT.

9) Pruebas sugeridas

- Crear 3 comunidades (A, C, B) y verificar orden (A, B, C) mediante IC_ListarComunidadesOrdenadas.
- Publicar mensajes en 'B' (2 mensajes) y 'A' (1 mensaje); verificar NumMensajes actualizado.
- Buscar comunidad inexistente al publicar: verificar error/retorno False.
- Generar IC_ReporteComunidades y validar formato esperado.
- Generar DOT y renderizar con Graphviz (dot -Tpng archivo.dot -o bst.png).

10) Riesgos, acuerdos y convenciones

- Evitar duplicar el modelo de 'Comunidad'. El BST sólo indexa; la lista es la fuente de verdad.
- Unificar nombres de unidad: usar ArbolBST (Jairo/Asher). Si se usa ComunidadesBST de Mario, renombrar a ArbolBST_Compat y encapsular.
- Convención de fechas: ISO 'yyyy-mm-dd hh:nn:ss' para interoperabilidad.
- Todas las inserciones de comunidades deben pasar por IC_CrearComunidad para mantener el índice en sincronía.
- Evitar referencias colgantes: si se elimina una comunidad, primero quitarla del índice, luego de la lista base.

11) Anexos — Fragmentos clave

11.1 Firma de tipos (Jairo/Asher) — ArbolBST

```
type
  PBSNodo = ^TBSNodo;
  TBSNodo = record
    Clave: string;           // Nombre de la comunidad (KEY)
    Ref: PComunidadNodo;    // Referencia a la lista base
    Izq, Der: PBSNodo;
  end;
```

11.2 API pública (Jairo/Asher) — ArbolBST

```
procedure BST_Insert(ComRef: PComunidadNodo);
function BST_Find(const Nombre: string): PComunidadNodo;
procedure BST_FillNamesInOrder(Dst: TStrings);
function BST_ObtenerComunidadesOrdenadas: TStringList;
procedure BST_CollectInOrderRefs(List: TList);
```

11.3 Firma de tipos (Mario) — ComunidadesBST

```
type
  PMensaje = ^TMensaje;
  TMensaje = record
    correo, contenido: String;
    fecha: TDateTime;
    sig: PMensaje;
  end;

  PComunidad = ^TComunidad;
  TComunidad = record
    nombre: String;
    fecha_creacion: TDateTime;
    num_mensajes: LongInt;
    mensajes: PMensaje;
    left, right: PComunidad;
  end;
```

11.4 Emisión DOT (Mario) — idea a reutilizar

```
procedure BST_ToDOT(root: PComunidadBST; const pathDot: String);
// Adaptable para el índice actual recorriendo referencias y emitiendo etiquetas.
```